

[All Collections](#) > [API](#) > [Access Token Migration Guide](#)

Access Token Migration Guide

Migrate your legacy access tokens to our current token model to use our most up to date API features

Updated over a year ago

[Overview](#)[Requirements for new applications](#)[Migration Path](#)

Overview

NationBuilder's v2 API endpoints require the use of our current tokens. Unlike our original (legacy) access tokens, current tokens expire after 24 hours and can only be granted via the OAuth authorization code flow.

In the future, we will deprecate legacy tokens and **all client applications will be required to use the current token model.**

Prior to this deprecation, developers can migrate their applications to new access tokens on a per application basis. The current tokens are, and will continue to be, backwards compatible with all API versions.

Requirements for new applications

Your application must use the authorization code OAuth flow and implement a refresh token flow before you can migrate to our current model of access token. If not, your application will encounter 401s because these tokens expire after 24 hours. Please see the [API Authentication Quickstart Guide](#) for instructions on implementing the authorization code and refresh flows.

Migration Path

This migration path allows you to exchange your legacy tokens for new tokens via the refresh flow. NationBuilder Admins will not have to re-auth their nations for your application and there should be no disruption to your service.

 *Note: The new tokens you receive through these steps will not have the same access token and refresh token value. Setting the refresh tokens is a one-time migration update; you should not manually change your access or refresh tokens again.*

 **Please implement the following very carefully.**

1. Ensure that your application is using the OAuth authorization code flow with a refresh flow. If you have implemented a refresh flow, you should have also implemented an association between an access token and its refresh token.
2. Store the legacy access token value as the refresh token value for each of the valid access tokens in your system. In other words, make each access token's refresh token associated with it the same as the access token.

For example, if your system has three access tokens:

```
{ "access_token": "asdfasdfasdf", "refresh_token": null }, { "access_token":
```

You should update them to:

```
{ "access_token": "asdfasdfasdf", "refresh_token": "asdfasdfasdf" }, { "acce
```

3. If your application is using the OAuth authorization code flow, a refresh flow, and you've updated your access tokens' refresh tokens, you are ready to convert to new access tokens.
4. Navigate to **Settings > Developer > Your apps**. Edit the application you are converting to current tokens.

 *Note: It is **critical** that you do not proceed to the following step until you have completed steps 1-4.*

5. You will see the "Convert to current access tokens" section at the top of the page. **It is critical that you do not flip this switch until *after* you have completed all of the steps above.** When you're sure you're ready, click it and your application will be updated immediately.

That's it! All of your existing access tokens will be set to expire in 24 hours from the time they were created. That means any access tokens created more than 24 hours ago will expire immediately.

Your application should go through the refresh token flow with these tokens when they expire. The `refresh_token` values you set in step 2 above will be valid to use as refresh tokens to get new access tokens in return.

All of your legacy tokens can be exchanged for current tokens this way, and all future tokens issued to your application will be of the current token model.

Related Articles

- API Authentication Quick Start guide >
- Generating API Tokens >
- NationBuilder API QuickStart Guide >
- API Authentication Guide >

Did this answer your question?





[Demos & webinars](#) [Courses](#) [Website themes](#) [API documentation](#)